DISCOVERING AI Agent

Tìm hiểu tác tử trí tuệ nhân tạo

Triệu Tuấn Anh MSSV B23DCCN053

1 Hiểu đúng về AI Agent

Giả sử một sinh viên muốn tìm buổi họp nhóm phù hợp với lịch học của bốn người. Chương trình chỉ trả lời cách đặt lịch chưa hoàn thành công việc. Một tác tử được cấp quyền phù hợp có thể đọc lịch, tìm giờ trống, đề xuất phương án và tạo lời mời sau khi được xác nhận. Ví dụ giả định này giúp phân biệt việc tạo câu trả lời với việc thực hiện một nhiệm vụ.

Google Cloud nhấn mạnh tác tử theo đuổi mục tiêu và hành động thay người dùng [1]. IBM chú ý đến tổ chức công việc và sử dụng công cụ [2]. McKinsey nhìn tác tử như một thành phần có thể phối hợp với các thành phần khác trong quy trình [3]. Từ đó, tôi hiểu AI Agent là một hệ thống nhận thông tin, lựa chọn hành động và kiểm tra kết quả để tiến tới mục tiêu trong phạm vi được giao.

Mục tiêu và quyền hạn cần được tách biệt

Trong ví dụ họp nhóm, mục tiêu là tìm giờ phù hợp; dữ liệu là lịch học; công cụ là thao tác đọc lịch hoặc tạo sự kiện. Quyền đọc lịch không tự động bao gồm quyền gửi lời mời. Nếu chưa được phép gửi, tác tử phải dừng ở bản đề xuất. Vì vậy, tính tự chủ nên được hiểu là khả năng tự chọn các bước trong giới hạn, không phải quyền thực hiện mọi hành động.

Một mô hình dự đoán thời gian trống có thể là thành phần của tác tử, nhưng bản thân dự đoán chưa tạo thành toàn bộ hệ thống. Ngược lại, có thể xây dựng tác tử đơn giản bằng luật hoặc bảng giá trị mà không cần mô hình ngôn ngữ lớn. Điều quan trọng khi mô tả một tác tử là xác định nó quan sát gì, được chọn hành động nào và dùng tiêu chí nào để biết nhiệm vụ đã xong.

Cách hiểu dùng trong bài

Tôi xem mô hình là bộ phận đưa ra dự đoán hoặc gợi ý, còn tác tử là hệ thống sử dụng kết quả đó để quyết định và hành động. Khi đánh giá tác tử họp nhóm, cần kiểm tra giờ có xung đột không và lời mời đã được xác nhận chưa; câu trả lời trôi chảy không đủ chứng minh nhiệm vụ thành công. Cách nhìn này cũng áp dụng cho tác tử mê cung ở phần thực hành: tiêu chí là tới đích hợp lệ, không phải chỉ vẽ ra một đường.

2 Cấu tạo và vòng lặp hoạt động

Có thể đọc một tác tử qua sáu thành phần: mục tiêu, thông tin quan sát, bộ phận chọn hành động, bộ nhớ, công cụ và phản hồi. Trong mô tả của Google Cloud, vai trò, bộ nhớ, công cụ và mô hình là các yếu tố đáng chú ý [1]. Tuy nhiên, một ví dụ nhỏ không bắt buộc phải có đầy đủ mọi loại bộ nhớ hoặc công cụ phức tạp.

Mục tiêu → Quan sát → Quyết định → Hành động → Kiểm tra

Sau bước kiểm tra, hệ thống có thể kết thúc, hỏi thêm hoặc quay lại quan sát. Vòng lặp chỉ có ý nghĩa khi thông tin mới có thể làm thay đổi quyết định. Chẳng hạn, một giờ họp vừa được người khác đặt phải bị loại khỏi các lựa chọn tiếp theo. Nếu tác tử tiếp tục gửi cùng yêu cầu mà không xét kết quả, nó chỉ đang lặp thao tác chứ chưa thích nghi tốt.

Diễn giải bằng ví dụ đặt lịch

Thành phần	Ý nghĩa trong tình huống giả định
Mục tiêu	Tìm một giờ họp không trùng lịch học
Quan sát	Các khoảng bận và phản hồi của thành viên
Quyết định	Chọn giờ, hỏi thêm hoặc báo chưa tìm được
Bộ nhớ	Các phương án đã thử và yêu cầu đã xác nhận
Công cụ	Đọc lịch, kiểm tra xung đột, tạo bản nháp lời mời
Phản hồi	Kết quả kiểm tra lịch và xác nhận của người dùng

Tôi phân biệt bộ nhớ làm việc với việc học lại mô hình. Ghi lại giờ đã bị từ chối giúp tác tử tránh hỏi lặp trong phiên hiện tại, nhưng chưa chứng minh các trọng số của mô hình đã thay đổi. Muốn nói hệ thống học được từ kinh nghiệm, cần chỉ ra cơ chế cập nhật và cách kiểm tra thay đổi đó. Sự phân biệt này giúp tránh dùng từ học cho mọi thao tác lưu thông tin.

Khi nào vòng lặp được dừng

Với ví dụ trên, có ba điểm dừng hợp lý: đã tìm được phương án được chấp nhận; không còn giờ phù hợp; hoặc cần quyền mà tác tử chưa được cấp. Có thể đặt thêm số lần thử tối đa để tránh hỏi và gọi công cụ liên tục. Đây là quy tắc thiết kế tôi đề xuất cho tình huống minh họa, không phải kết quả đo từ một hệ thống đặt lịch thực tế.

3 Các kiểu tác tử và cách phối hợp

IBM giới thiệu năm kiểu tác tử: phản xạ đơn giản, phản xạ dựa trên mô hình, hướng mục tiêu, dựa trên độ hữu ích và có khả năng học [2]. Có thể hiểu chúng qua cách lựa chọn hành động. Bảng dưới dùng các ví dụ tự xây dựng trong một thư viện để làm rõ sự khác nhau; không hàm ý thư viện thực tế đã triển khai các hệ thống này.

Kiểu tác tử	**Cách chọn hành động**	**Ví dụ minh họa**
Phản xạ đơn giản	Áp dụng luật cho quan sát hiện tại	Báo hết chỗ khi số ghế trống bằng không
Dựa trên mô hình	Dùng quan sát và trạng thái đã ghi nhớ	Robot nhớ khu vực vừa kiểm tra
Hướng mục tiêu	Tìm chuỗi bước đưa đến đích	Tìm đường tới giá sách được yêu cầu
Dựa trên độ hữu ích	Cân nhắc nhiều tiêu chí giữa các cách đạt đích	Chọn đường vừa ngắn vừa ít cản trở người đọc
Có khả năng học	Cải thiện cách chọn hành động từ kinh nghiệm	Điều chỉnh lựa chọn sau các lượt di chuyển

Các kiểu trên không nên được hiểu như một bảng xếp hạng bắt buộc. Nếu chỉ cần kiểm tra còn ghế hay không, một luật rõ ràng dễ kiểm thử hơn việc thêm mô hình lớn. Nếu cần chọn đường qua nhiều ngã rẽ, hệ thống phải xét ảnh hưởng của các bước tiếp theo. Khả năng phù hợp với nhiệm vụ quan trọng hơn số lượng thành phần được gắn vào tác tử.

Một tác tử và nhiều tác tử

Cách chia một hoặc nhiều tác tử nói về tổ chức hệ thống, không trùng với cách phân loại theo năng lực. Một tác tử có thể xử lý toàn bộ yêu cầu; nhiều tác tử có thể chia vai trò và trao đổi kết quả. Trong ví dụ giả định tìm sách, một tác tử tra danh mục, một tác tử kiểm tra vị trí và một tác tử tổng hợp hướng dẫn cho người đọc.

Tách vai trò có ích khi từng bước có dữ liệu hoặc trách nhiệm khác nhau. Tuy nhiên, nếu danh mục ghi sai mã sách, nhiều tác tử cùng dựa vào mã đó vẫn có thể trả lời sai. Tôi sẽ yêu cầu kết quả trung gian mang theo mã sách và thời điểm kiểm tra, thay vì chỉ truyền một câu kết luận. Thêm tác tử không thay thế được việc xác thực dữ liệu và kiểm tra kết quả cuối cùng.

Bài thực hành chỉ xây dựng một tác tử học từ tương tác. Bảng Q là cách lưu kinh nghiệm chọn hành động, không phải một tác tử thứ hai và cũng không phải mô hình ngôn ngữ.

4 Ứng dụng và cách xác định giá trị

McKinsey bàn đến nhiều cách đưa tác tử vào công việc, từ hỗ trợ cá nhân và tự động hóa quy trình đến các hệ thống chuyên biệt hoặc vai trò làm việc độc lập hơn [3]. Tôi hiểu điểm cần đánh giá là phần việc nào được hoàn thành tốt hơn, chứ không chỉ việc sản phẩm có thêm một cửa sổ trò chuyện.

Tình huống hỗ trợ tra cứu lịch thi

Xét một tác tử giả định giúp sinh viên tra lịch thi. Đầu vào là mã học phần và lớp; nguồn thông tin là lịch thi được công bố. Tác tử cần tìm đúng kỳ thi, kiểm tra phiên bản lịch rồi trả lại phòng, giờ và đường dẫn thông báo. Nếu chưa có mã lớp, nó nên hỏi lại thay vì tự chọn một lớp có tên gần giống. Đây là ví dụ thiết kế để phân tích, không phải ứng dụng đã được triển khai.

Ở mức hỗ trợ cá nhân, tác tử chỉ tóm tắt lịch để người dùng tự nhập vào lịch riêng. Ở mức tự động hóa quy trình, nó có thể tạo bản nháp sự kiện và chờ xác nhận. Nếu hệ thống được tích hợp sâu hơn, cần xác định ai chịu trách nhiệm khi lịch đổi và cách thông báo bản cập nhật. Cùng một mục tiêu có thể dẫn đến phạm vi tự chủ rất khác nhau.

Câu hỏi đánh giá	**Dấu hiệu có thể kiểm tra**
Kết quả có đúng không	Mã học phần, lớp và thời gian khớp thông báo
Thông tin có còn hiệu lực không	Có phiên bản hoặc thời điểm công bố
Tác tử có vượt quyền không	Không tự gửi thông báo nếu chưa được phép
Có tiết kiệm thao tác không	Đếm bước người dùng phải làm trước và sau
Khi thiếu dữ liệu thì sao	Hỏi lại hoặc báo chưa xác nhận được

Lợi ích phải đi kèm điều kiện

Ví dụ này cho thấy giảm thao tác chỉ có giá trị khi kết quả đúng. Một hệ thống tạo lịch rất nhanh nhưng chọn sai lớp có thể làm người dùng mất thêm thời gian sửa. Nếu muốn tuyên bố hiệu quả, tôi cần so sánh trên cùng tập yêu cầu và ghi cả lỗi, thời gian chờ lẫn số lần phải nhờ con người hỗ trợ. Bài này không tự đưa ra số phần trăm tăng năng suất vì chưa có phép đo như vậy.

Ba bài đọc gợi ra ba câu hỏi bổ sung cho nhau: tác tử làm được gì, cơ chế nào cho phép nó làm điều đó và tổ chức cần kiểm soát việc sử dụng ra sao. Một bản trình diễn có thể trả lời câu hỏi đầu; để sử dụng ổn định còn phải trả lời hai câu hỏi sau.

5 Rủi ro và vai trò của con người

Tác tử có thể nhận dữ liệu thiếu, chọn công cụ sai hoặc tiếp tục thực hiện một kế hoạch không còn phù hợp. IBM nêu các rủi ro như vòng lặp phản hồi, phụ thuộc giữa tác tử và quyền riêng tư [2]. Tôi cho rằng quyền hành động càng rộng thì càng cần xác định trước điểm dừng và người chịu trách nhiệm kiểm tra.

Rủi ro trong tình huống tra lịch thi

Nếu tác tử đọc một thông báo cũ, câu trả lời vẫn có thể đủ phòng và giờ nhưng không còn đúng. Nếu gặp hai thông báo khác nhau, việc tự chọn bản thuận tiện hơn sẽ che giấu sự không chắc chắn. Một cách xử lý tôi đề xuất là giữ đường dẫn và thời điểm của thông báo, ưu tiên bản được xác nhận có hiệu lực, còn trường hợp mâu thuẫn thì báo rõ để người dùng kiểm tra.

Vấn đề	**Biện pháp thiết kế đề xuất**
Thiếu hoặc mâu thuẫn thông tin	Hỏi lại, đối chiếu nguồn, không tự bịa phần còn thiếu
Gọi công cụ lặp mãi	Giới hạn số lần thử và cho phép người dùng dừng
Thực hiện hành động quá phạm vi	Tách quyền đọc, tạo bản nháp và gửi chính thức
Khó xác định bước làm sai	Ghi nhật ký công cụ, thời điểm và kết quả kiểm tra
Lộ thông tin người dùng	Chỉ lấy dữ liệu cần thiết và giới hạn người được xem

Nhật ký hữu ích cần ghi được hệ thống đã gọi công cụ nào và nhận kết quả gì. Không nhất thiết phải công bố toàn bộ diễn giải nội bộ của mô hình để xác minh một thao tác. Với tác tử lịch thi, bằng chứng quan trọng là thông báo đã được đọc và sự kiện nào đã được tạo. Cách ghi này giúp người dùng kiểm tra hành động thực tế thay vì chỉ tin vào lời giải thích của tác tử.

Con người quyết định phạm vi tự chủ

Đối với một yêu cầu tra cứu, có thể cho tác tử tự đọc thông báo công khai. Đối với việc gửi thay đổi đến nhiều người, cần bước xác nhận rõ ràng. Khi thông tin chưa đủ, báo chưa xác nhận được là một kết quả hợp lệ. Tôi sẽ không coi việc luôn trả lời hoặc luôn tiếp tục hành động là tiêu chí tốt hơn việc dừng đúng lúc.

Từ tình huống minh họa, tiêu chí sử dụng có trách nhiệm gồm tính đúng đắn của kết quả, tuân thủ quyền hạn và khả năng sửa sai. Một tác tử hữu ích phải giúp người dùng biết nó đã làm gì, phần nào chưa chắc và bước nào vẫn cần quyết định của con người.

6 Tổng hợp và liên hệ với học tăng cường

Sau khi đọc các tài liệu, tôi rút ra ba điểm: tác tử cần mục tiêu có thể kiểm tra; hành động phải dựa trên thông tin và quyền được cấp; phản hồi phải có khả năng thay đổi lựa chọn tiếp theo. Một hệ thống tạo câu trả lời hay nhưng không kiểm tra kết quả thực hiện vẫn chưa giải quyết đầy đủ nhiệm vụ của người dùng.

Học tăng cường minh họa điều gì

Trong học tăng cường, tác tử thử hành động trong môi trường và nhận phần thưởng. Chính sách là cách chọn hành động; bảng Q ghi giá trị ước lượng của từng lựa chọn [4]. Ví dụ mê cung ở phần sau làm rõ vòng tương tác này bằng một không gian nhỏ, quan sát được. Việc cập nhật Q là học từ trải nghiệm, khác với chỉ ghi nhớ nội dung một cuộc trò chuyện.

Khái niệm	**Trong bài mê cung**
Tác tử	Bộ phận chọn hướng đi
Môi trường	Lưới đường đi và tường
Trạng thái	Tọa độ hàng và cột hiện tại
Hành động	Trái, phải, lên hoặc xuống
Phản hồi	Phần thưởng và trạng thái tiếp theo
Chính sách	Cách chọn hướng từ bảng Q và mức khám phá

Ban đầu, tác tử chưa biết lựa chọn nào tốt nên phải khám phá. Khi đã có kinh nghiệm, nó khai thác lựa chọn có giá trị Q cao. Trong bài này, mỗi bước đi mất một đơn vị thưởng, va tường bị phạt và tới đích được thưởng. Cách đặt mục tiêu đó khuyến khích một đường đi hợp lệ, ngắn hơn các đường vòng. Tuy nhiên, thiết kế phần thưởng không tự chứng minh rằng lượt học cụ thể đã tìm được đường ngắn nhất.

Giới hạn của minh họa

Mê cung cố định khác xa một tác tử dùng công cụ trong doanh nghiệp: không có ngôn ngữ tự nhiên, dữ liệu riêng tư hoặc tác tử khác. Vì vậy, kết quả chỉ minh họa cách một learning agent cải thiện quyết định trong môi trường đã định nghĩa. Không thể dùng một đường đi thành công để kết luận mọi AI Agent đều cần Q-learning hoặc có thể xử lý nhiệm vụ thực tế an toàn.

Trong thực hành, tôi sẽ chạy notebook thật, kiểm tra đường đi không xuyên tường và dùng BFS làm đối chứng độc lập cho độ dài tối thiểu. BFS không cung cấp đáp án cho quá trình học. Cách kiểm tra này nối phần lý thuyết với bằng chứng quan sát được: tác tử đã hành động ra sao và kết quả có đạt tiêu chí đã đặt hay không.

PHẦN THỰC HÀNH

7 Thiết lập notebook và môi trường

Thực hành Q-learning trên mê cung 10 × 10 theo ví dụ GeeksforGeeks [4]. Mã được viết lại thành các hàm nhỏ; giữ ma trận, bốn hướng di chuyển, phần thưởng và lịch giảm epsilon của ví dụ. Notebook chạy bằng Python 3.11 trong môi trường WSL hiện có, dùng NumPy trên CPU.

Hình 1 Cell khởi tạo và output kiểm tra môi trường

In [1]:
```
import json, platform
from collections import deque
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap
%matplotlib inline

plt.rcParams.update({"text.color": "black", "axes.labelcolor": "black",
                     "xtick.color": "black", "ytick.color": "black",
                     "font.size": 12, "figure.facecolor": "white"})
MAZE = np.array([
    [0, 1, 1, 1, 1, 1, 1, 1, 1, 1],
    [0, 0, 0, 0, 1, 0, 0, 0, 0, 1],
    [1, 1, 1, 0, 1, 0, 1, 1, 0, 1],
    [1, 0, 0, 0, 0, 0, 1, 0, 0, 1],
    [1, 0, 1, 1, 1, 1, 1, 0, 1, 1],
    [1, 0, 1, 0, 0, 0, 0, 0, 1, 1],
    [1, 0, 1, 0, 1, 1, 1, 0, 1, 1],
    [1, 0, 1, 0, 1, 0, 0, 0, 1, 1],
    [1, 0, 1, 0, 1, 0, 1, 0, 0, 1],
    [1, 1, 1, 0, 1, 1, 1, 1, 0, 0],
], dtype=np.int8)
START, GOAL = (0, 0), (9, 9)
ACTIONS = ((0, -1), (0, 1), (-1, 0), (1, 0))
print("Python:", platform.python_version(), "| NumPy:", np.__version__)
print("Maze:", MAZE.shape, "| Free cells:", int((MAZE == 0).sum()))
print("Start:", START, "| Goal:", GOAL, "| Q shape:", MAZE.shape + (4,))
```

```
Python: 3.11.17 | NumPy: 2.4.6
Maze: (10, 10) | Free cells: 43
Start: (0, 0) | Goal: (9, 9) | Q shape: (10, 10, 4)
```

Mê cung có 43 ô đi được. Bảng Q có kích thước 10 × 10 × 4, tương ứng 400 giá trị; các phần tử khởi tạo bằng không. Điểm bắt đầu là (0, 0), đích là (9, 9). Mỗi tọa độ viết theo thứ tự hàng rồi cột.

Thiết lập: 5.000 lượt, seed 42, alpha 0,1, gamma 0,9; epsilon từ 0,5 giảm theo hệ số 0,995 và không dưới 0,01. Tới đích nhận +50; va tường hoặc ra ngoài nhận −10; bước thường nhận −1.

8 Môi trường và cập nhật kinh nghiệm

Hàm valid kiểm tra ô đi được. Hàm transition trả trạng thái tiếp theo, phần thưởng và cờ kết thúc. Va tường hoặc ra ngoài kết thúc lượt; trạng thái được giữ tại ô trước va chạm, như quy ước của ví dụ [4].

Hình 2 Hàm môi trường và ba output kiểm tra

In [2]
```
def valid(maze, state):
    row, col = state
    return bool(0 <= row < maze.shape[0] and 0 <= col < maze.shape[1] and maze[state]
== 0)

def transition(maze, state, action, goal):
    delta = ACTIONS[action]
    following = (state[0] + delta[0], state[1] + delta[1])
    if not valid(maze, following):
        return state, -10, True
    if following == goal:
        return following, 50, True
    return following, -1, False
print("Down from start:", transition(MAZE, START, 3, GOAL))
print("Wall from start:", transition(MAZE, START, 1, GOAL))
print("Goal transition:", transition(MAZE, (9, 8), 1, GOAL))
```

```
Down from start: ((1, 0), -1, False)
Wall from start: ((0, 0), -10, True)
Goal transition: ((9, 9), 50, True)
```

choose_action cân bằng thử ngẫu nhiên và chọn giá trị Q lớn nhất. update_q điều chỉnh giá trị của lựa chọn vừa thực hiện. Nếu lượt đã kết thúc, giá trị tương lai bằng không; không cộng thêm giá trị ở trạng thái đích.

Hình 3 Chọn hành động và kiểm tra cập nhật Q ở đích

In [3]
```
def choose_action(q, state, epsilon, rng):
    if not 0 <= epsilon <= 1:
        raise ValueError("epsilon must be between zero and one")
    if rng.random() < epsilon:
        return int(rng.integers(4))
    return int(np.argmax(q[state]))

def update_q(q, state, action, following, reward, done, alpha=0.1, gamma=0.9):
    future = 0.0 if done else float(np.max(q[following]))
    old = q[state + (action,)]
    q[state + (action,)] = old + alpha * (reward + gamma * future - old)
probe = np.zeros((10, 10, 4))
update_q(probe, (9, 8), 1, GOAL, 50, True)
print("Terminal update with alpha=0.1:", probe[9, 8, 1])
print("alpha=0.1 | gamma=0.9 | epsilon: 0.5 -> 0.01")
```

```
Terminal update with alpha=0.1: 5.0
alpha=0.1 | gamma=0.9 | epsilon: 0.5 -> 0.01
```

9 Huấn luyện tác tử

Mỗi lượt bắt đầu lại từ START. Tác tử chọn hành động, nhận phản hồi rồi cập nhật Q. Seed cố định giúp chạy lại có cùng kết quả. Giới hạn 400 bước mỗi lượt là chốt an toàn được bổ sung; lượt đạt giới hạn được ghi là truncated, không bị coi là đã tới đích.

Hình 4 Toàn bộ hàm train và output sau 5.000 lượt

In [4]

```
def train(maze=MAZE, start=START, goal=GOAL, episodes=5000, seed=42, max_steps=400):
    if episodes <= 0 or max_steps <= 0 or not valid(maze, start) or not valid(maze, g
oal):
        raise ValueError("Positive budgets and valid endpoints are required")
    rng = np.random.default_rng(seed)
    q = np.zeros(maze.shape + (4,), dtype=np.float64)
    epsilon = 0.5
    returns, lengths, successes = [], [], []
    truncated = 0
    for _ in range(episodes):
        state, total, reached = start, 0, False
        for step in range(1, max_steps + 1):
            action = choose_action(q, state, epsilon, rng)
            following, reward, done = transition(maze, state, action, goal)
            update_q(q, state, action, following, reward, done)
            state, total = following, total + reward
            if done:
                reached = state == goal
                break
        else:
            truncated += 1
        returns.append(total)
        lengths.append(step)
        successes.append(reached)
        epsilon = max(0.01, epsilon * 0.995)
    return {"q": q, "returns": np.array(returns), "lengths": np.array(lengths),
            "successes": np.array(successes), "truncated": truncated, "epsilon": epsi
lon}
result = train(episodes=5000, seed=42, max_steps=400)
Q = result["q"]
for left in (0, 1000, 4000):
    right = left + 1000
    success = result["successes"][left:right].mean()
    mean_return = result["returns"][left:right].mean()
    print(f"Episodes {left+1:4d}-{right}: success={success:.1%}, return={mean_retur
n:.2f}")
print("Final epsilon:", result["epsilon"], "| Truncated:", result["truncated"])
```

```
Episodes    1-1000: success=25.0%, return=-10.59
Episodes 1001-2000: success=87.0%, return=20.65
Episodes 4001-5000: success=87.6%, return=20.95
Final epsilon: 0.01 | Truncated: 0
```

Các mốc trong output là thống kê từ lượt thực thi notebook. Không có lượt nào bị cắt bởi giới hạn bước. Epsilon giảm dần tới 0,01, nên kể cả cuối huấn luyện tác tử vẫn thỉnh thoảng chọn hành động khám phá.

10 Kiểm tra tác tử sau khi học

Đánh giá dưới đây tắt khám phá và luôn chọn argmax của Q. Hàm greedy_path không lọc trước hướng va tường và không tự tìm đường thay tác tử. Nếu chính sách va chạm, lặp hoặc quá số bước, hàm báo đúng trạng thái đó thay vì ép đường đi tới đích.

Hình 5 Chính sách greedy và đường đi thực tế

In [5]
```
def greedy_path(maze, q, start, goal, max_steps=200):
    path, visited = [start], {start}
    for _ in range(max_steps):
        if path[-1] == goal:
            return path, "goal"
        following, _, done = transition(maze, path[-1], int(np.argmax(q[path[-1]])),
goal)
        if done and following != goal:
            return path, "collision"
        if following in visited:
            return path, "loop"
        path.append(following)
        visited.add(following)
    return path, "goal" if path[-1] == goal else "step_limit"
path, status = greedy_path(MAZE, Q, START, GOAL)
print("Greedy status:", status, "| Moves:", len(path) - 1)
print("Learned route:", path)
```

```
Greedy status: goal | Moves: 24
Learned route: [(0, 0), (1, 0), (1, 1), (1, 2), (1, 3), (2, 3), (3, 3), (3, 4), (3, 5), (2,
5), (1, 5), (1, 6), (1, 7), (1, 8), (2, 8), (3, 8), (3, 7), (4, 7), (5, 7), (6, 7), (7, 7),
(8, 7), (8, 8), (9, 8), (9, 9)]
```

Kết quả đánh giá	**Giá trị thực thi**
Trạng thái cuối	goal
Số bước di chuyển	24
Tổng thưởng không chiết khấu	27

Đường đi có 25 tọa độ, tương ứng 24 lần di chuyển. Có 23 bước thường nhận −1 và bước cuối tới đích nhận +50, nên tổng thưởng là 27. Tổng thưởng này không phải giá trị Q: Q được học với gamma 0,9 nên có xét chiết khấu phần thưởng tương lai.

Chỉ nhìn đường vẽ chưa đủ kết luận ngắn nhất. Cần kiểm từng bước không xuyên tường, đúng bốn hướng được phép và kết thúc tại GOAL. Trang sau dùng một thuật toán độc lập để kiểm tra thêm độ dài tối thiểu.

11 Đối chiếu độc lập bằng BFS

BFS duyệt lưới không trọng số theo từng lớp khoảng cách, chỉ được gọi sau khi Q-learning đã học xong. Hàm này không sửa Q, không cung cấp hành động cho train và không được dùng để thay đường đi greedy.

Hình 6 Mã BFS và output đối chiếu đường đi

In [6]:

```
def shortest_path(maze, start, goal):
    pending, parent = deque([start]), {start: None}
    while pending:
        state = pending.popleft()
        if state == goal:
            path = []
            while state is not None:
                path.append(state)
                state = parent[state]
            return path[::-1]
        for delta in ACTIONS:
            following = (state[0] + delta[0], state[1] + delta[1])
            if valid(maze, following) and following not in parent:
                parent[following] = state
                pending.append(following)
    return []

def valid_path(maze, path):
    if not path or not all(valid(maze, state) for state in path):
        return False
    return all(abs(a[0] - b[0]) + abs(a[1] - b[1]) == 1 for a, b in zip(path, path
[1:]))
reference = shortest_path(MAZE, START, GOAL)
assert status == "goal" and valid_path(MAZE, path)
assert len(path) == len(reference)
print("BFS minimum moves:", len(reference) - 1)
print("Valid route:", valid_path(MAZE, path), "| Same length:", len(path) == len(refe
rence))
print("Greedy undiscounted return:", 50 - (len(path) - 2))
```

```
BFS minimum moves: 24
Valid route: True | Same length: True
Greedy undiscounted return: 27
```

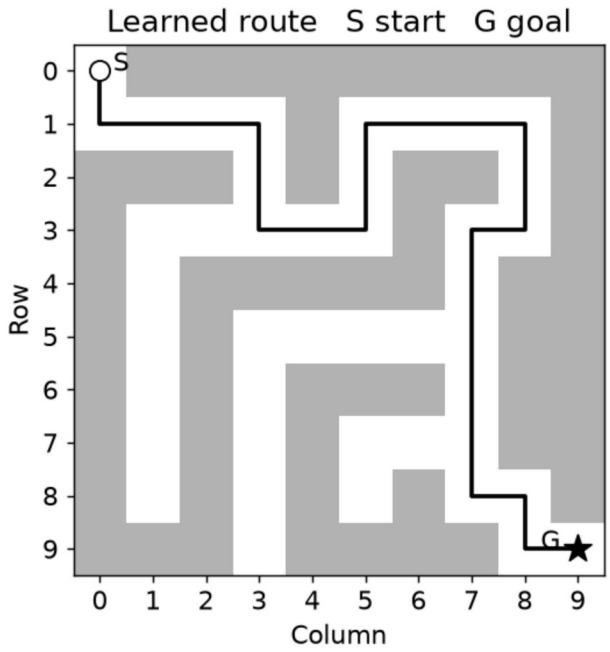
valid_path kiểm tra các ô đều đi được và hai tọa độ liên tiếp kề nhau theo hàng hoặc cột. Output xác nhận đường đi hợp lệ và độ dài Q-learning bằng BFS: đều 24 bước. Vì vậy có thể kết luận đường đi greedy là ngắn nhất trên đúng mê cung cố định này, không khái quát sang mọi mê cung hoặc mọi seed.

12 Quan sát đường đi học được

Hình dưới là output do notebook tự vẽ từ biến path, không phải hình lấy từ bài hướng dẫn. Ô trắng là đường đi, ô xám là tường; đường đen nối các hành động đã chọn. S và G lần lượt đánh dấu đầu và đích.

Hình 7 Code vẽ và output đường đi sau huấn luyện

In [7]:
```
fig, ax = plt.subplots(figsize=(5, 4.6), dpi=130)
ax.imshow(MAZE, cmap=ListedColormap(["white", "0.7"]), vmin=0, vmax=1)
rows, columns = zip(*path)
ax.plot(columns, rows, color="black", linewidth=2)
ax.scatter(START[1], START[0], marker="o", s=90,
           facecolors="white", edgecolors="black", zorder=4)
ax.scatter(GOAL[1], GOAL[0], marker="*", s=180, color="black", zorder=4)
ax.set(xticks=range(10), yticks=range(10), xlabel="Column", ylabel="Row")
ax.set_title("Learned route   S start   G goal")
ax.text(START[1]+0.25, START[0], "S", color="black")
ax.text(GOAL[1]-0.7, GOAL[0], "G", color="black")
fig.tight_layout()
plt.show()
```



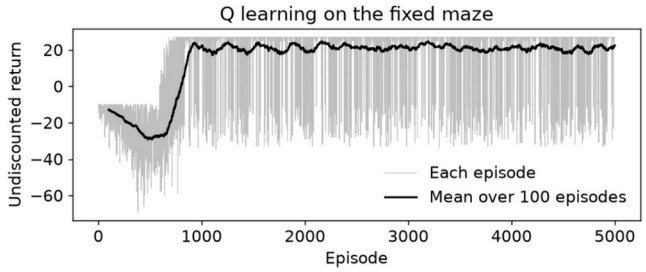
Trục Row tăng từ trên xuống đúng quy ước của ma trận. Đường đi được vẽ qua tâm các ô hợp lệ và đi qua những đoạn bắt buộc của mê cung. Đây là minh họa trực quan cho kết quả đã được kiểm bằng valid_path và BFS, không phải bằng chứng duy nhất để khẳng định thành công.

13 Phần thưởng và tổng hợp output

Đường xám ghi tổng thưởng từng lượt; đường đen là trung bình trượt 100 lượt. Đường cong có dao động vì huấn luyện vẫn khám phá. Không diễn giải một đường trung bình tăng thành bảo đảm mọi lượt đều thành công.

Hình 8 Code và output biểu đồ phần thưởng

In [8]:
```
fig, ax = plt.subplots(figsize=(7, 3.1), dpi=130)
returns = result["returns"]
ax.plot(np.arange(1, 5001), returns, color="0.75", linewidth=0.5,
        label="Each episode")
rolling = np.convolve(returns, np.ones(100)/100, mode="valid")
ax.plot(np.arange(100, 5001), rolling, color="black", linewidth=1.5,
        label="Mean over 100 episodes")
ax.set(xlabel="Episode", ylabel="Undiscounted return",
       title="Q learning on the fixed maze")
ax.legend(frameon=False)
fig.tight_layout()
plt.show()
```



Hình 9 Cell tổng hợp các kết quả thực thi

In [9]:
```
summary = {
    "seed": 42, "episodes": 5000, "q_shape": list(Q.shape),
    "free_cells": int((MAZE == 0).sum()), "greedy_status": status,
    "learned_moves": len(path)-1, "bfs_moves": len(reference)-1,
    "valid_route": valid_path(MAZE, path),
    "greedy_return": 50-(len(path)-2),
    "last_1000_success_rate": float(result["successes"][-1000:].mean()),
    "last_1000_mean_return": float(result["returns"][-1000:].mean()),
    "truncated_episodes": int(result["truncated"]),
    "execution_device": "CPU NumPy", "path": path,
}
print(json.dumps(summary))
```

```
{"seed": 42, "episodes": 5000, "q_shape": [10, 10, 4], "free_cells": 43, "greedy_status": "g
oal", "learned_moves": 24, "bfs_moves": 24, "valid_route": true, "greedy_return": 27, "last_
1000_success_rate": 0.876, "last_1000_mean_return": 20.951, "truncated_episodes": 0, "execut
ion_device": "CPU NumPy", "path": [[0, 0], [1, 0], [1, 1], [1, 2], [1, 3], [2, 3], [3, 3],
[3, 4], [3, 5], [2, 5], [1, 5], [1, 6], [1, 7], [1, 8], [2, 8], [3, 8], [3, 7], [4, 7], [5,
7], [6, 7], [7, 7], [8, 7], [8, 8], [9, 8], [9, 9]]}
```

Trong 1.000 lượt cuối, tỷ lệ tới đích là 87,6% và thưởng trung bình 20,951. Hai số này khác kết quả greedy 24 bước, thưởng 27 vì điều kiện chọn hành động khác nhau. Thí nghiệm chỉ chạy một seed trên một mê cung; chưa đánh giá khả năng tổng quát hóa.

TÀI LIỆU THAM KHẢO

[1] Google Cloud. What is an AI agent.

https://cloud.google.com/discover/what-are-ai-agents

Truy cập ngày 23/10/2026.

[2] IBM. What are AI agents. Tác giả Anna Gutowska.

https://www.ibm.com/think/topics/ai-agents

Truy cập ngày 23/10/2026.

[3] McKinsey and Company. What is an AI agent. Ngày công bố 25/03/2025.

https://www.mckinsey.com/featured-insights/mckinsey-explainers/what-is-an-ai-agent

Truy cập ngày 23/10/2026.

[4] GeeksforGeeks. Reinforcement Learning.

https://www.geeksforgeeks.org/machine-learning/what-is-reinforcement-learning/

Truy cập ngày 23/10/2026.

Ghi chú về phần thực hành

Mê cung, hành động, phần thưởng và lịch học được tham khảo từ [4]. Seed, giới hạn bước, kiểm tra terminal, đánh giá greedy nghiêm ngặt và đối chứng BFS được bổ sung cho việc kiểm chứng. Các hình code và output là ảnh chụp trình duyệt từ bản HTML xuất của notebook đã thực thi, với màu chữ được đặt đen; không dùng output mẫu của website.